

A World Model for Predictive Cyber Defence

Learning Network State Dynamics to Forecast Attacker Progression

SIH26153 — Technical Report

ABSTRACT

This report documents the design, architecture search, and evaluation of a self-supervised latent-transition model that learns from flow-level network telemetry and forecasts an attacker's progression through a 7-stage MITRE ATT&CK-derived taxonomy. The system is trained on three public intrusion-detection datasets (CIC-IDS-2018, CIC-IDS-2017, CTU-13) pooled into one 43-feature, 60-second-windowed representation, and evaluated with two supervised downstream heads: (i) binary infiltration-probability forecasting 1–5 minutes ahead, and (ii) 7-way stage classification. **The result that should be read as primary is a cross-episode holdout:** training on one of CIC-IDS-2018's two Infiltration episodes and testing purely on the other (never mixed) gives $AUC \approx 0.68\text{--}0.82$ — a real, above-chance, but modest generalization result, based on only the two such episodes the dataset contains. The commonly quoted pooled/in-distribution numbers ($AUC \approx 0.97\text{--}0.98$, obtained by training and testing on different time-windows of the *same* episodes) are reported for completeness and cross-architecture comparison only and should **not** be read as a generalization claim, since they can and likely do partly reflect episode-specific persistence rather than transferable signal. Roughly a dozen architectural variants were compared, including a Transformer-based model, a Recurrent State-Space Model (RSSM), domain-adversarial training, two independent graph-structured encoders, and a KL-bottleneck ablation. The strongest, seed-replicated configuration is an RSSM with an unfrozen encoder; unfreezing is shown here to improve the two *downstream classification/forecast heads* measured, which is a narrower claim than an improvement in the underlying latent dynamics themselves — this report's evaluation is centered on those two task heads (Section 3.4 states this boundary explicitly) and does not separately measure open-loop multi-step reconstruction error, future communication-graph structure, or other direct measures of world-model quality. Several stage classes have too few test occurrences to support a confident per-class claim (zero Reconnaissance/Exfiltration occurrences, 5–17 Lateral-Movement/Initial-Access/Impact occurrences at any forecast horizon); this is stated plainly rather than smoothed into the headline numbers. The system includes a SHAP-based explanation of single-window stage predictions (not of a full multi-step rollout — a narrower claim, stated explicitly in Section 7) and a working live-inference prototype (Flask backend + browser dashboard) accepting a CSV or raw PCAP file.

Contents

1. Problem Framing
2. Data and Feature Engineering
3. Core Architecture
4. Architecture Search
5. Per-Stage Forecasting Results
6. Baselines and the Cross-Episode Generalization Test

7. Explainability	
8. Working Prototype	
9. Known Limitations	
10. Conclusion	
Reproducibility · References (unnumbered, end of document)	

1. Problem Framing

The problem statement (SIH26153) asks for a system that learns network state dynamics from traffic telemetry, forecasts attacker progression, maps predictions onto MITRE ATT&CK stages, and exposes interpretable decision support — explicitly favouring a *world model* ($P(S_{t+1} | S_t)$, a model of how the network evolves) over a static classifier that only labels the present. Two properties of the task shaped every subsequent design decision:

- **Forecasting, not detection.** The deliverable is a probability of reaching a dangerous stage k steps into the future, not a same-timestep label. This rules out any architecture that only classifies the current window and pushes toward genuinely sequential models.
- **Interpretability is mandatory, not optional.** Both an attention/attribution channel (SHAP) and a closed-form, whiteboard-showable mechanism (a counted stage-transition matrix, Π) are required outputs, not an afterthought bolted on at the end.

2. Data and Feature Engineering

2.1 Datasets

Provenance, stated precisely to avoid any ambiguity: training used each dataset's already-flow-level export — CICFlowMeter's per-flow CSV output for CIC-IDS-2018/2017, and Argus's per-flow `binetflow` export for CTU-13 — re-aggregated by us from per-flow rows into a common 60-second-window schema. **No raw packet capture (PCAP) was processed for any of the three training datasets;** the public CIC-IDS-2018/2017 PCAP release is ~220 GB and was not downloaded (Section 9). The only component of this project that parses raw packets is the live-inference demo path (Section 8), which is separate from training and operates on user-supplied PCAP/CSV input at inference time only.

- **CIC-IDS-2018** (10 days, CICFlowMeter output) — the primary source; carries Reconnaissance, Initial Access, Lateral Movement, Command & Control, and Impact-stage attacks. No IP columns in the public release, which later blocked any host-communication graph representation for this dataset specifically.
- **CIC-IDS-2017** (8 days, `GeneratedLabelledFlows` release) — folded into training on explicit instruction; retains real `Source IP/Destination IP` columns, unlike CIC-IDS-2018.

- **CTU-13** (13 scenarios, Argus binetflow) — 13 independent real botnet captures (Neris, Rbot, Virut, Murlo, Sogou, and others), contributing mainly Benign/Command-and-Control labels; also carries real `SrcAddr/DstAddr` and real TTL.

Two datasets suggested by the problem statement were evaluated and **not** integrated, for stated, verified reasons (Section 9): CICIOT2023 (no timestamps, packets pre-shuffled into fixed-size groups — structurally incompatible with sequence forecasting) and raw PCAP-level capture for CIC-IDS-2018/2017 (the public PCAP release is ~220 GB, judged not worth the bandwidth/storage cost for the marginal features gained over the flow-level CSVs).

2.2 Stage-label mapping: provenance and scope

None of the three datasets ships native MITRE ATT&CK labels. The 7-stage taxonomy used throughout this report (Benign, Reconnaissance, Initial Access, Lateral Movement, Command and Control, Exfiltration, Impact) is obtained by keyword-matching each dataset's own native attack-type label string (e.g. “DDoS attacks-LOIC-HTTP”, “Infiltration”, “PortScan”) against a fixed, first-match-wins rule table (reproduced in full below), then assigning that single stage label to every flow in the window and taking the majority label as the window's stage. This mapping is a **first-class, reviewable modelling decision, not native ground truth**, and two consequences follow directly from that:

- **The 7 stages are treated as one label per window, in a fixed evaluation order** — real MITRE ATT&CK techniques and tactics can co-occur, repeat, and branch, and this project does not claim otherwise. The single-label-per-window taxonomy is an evaluation simplification chosen because the source datasets carry a single attack-type string per flow in the first place; it is not a modelling claim that real attacker behaviour is linear.
- **Two mapping decisions are judgment calls, stated here rather than left implicit in code:** CIC-IDS-2018's “Infiltration” label (its multi-phase exploit-delivery-plus-internal-portscan scenario) is mapped to *Lateral Movement* rather than *Initial Access*, on the basis that the labeled traffic is the attacker already inside and moving through the network; and CTU-13's “Background” traffic (unconfirmed ambient traffic, not attested as either attack or known-good) is mapped to *Benign* because it is not confirmed malicious, not because it is confirmed clean.

The full first-match-wins keyword table (`attack_map.py`).

Keyword (case-insensitive substring)	Stage
benign, normal, background	Benign
portscan, port scan, recon	Reconnaissance
bruteforce, brute force, patator, sql injection, xss, web attack	Initial Access
infil, heartbleed	Lateral Movement
bot, botnet	Command and Control
exfil	Exfiltration
ddos, dos(-/_/space), goldeneye, slowloris, slowhttptest, hulk	Impact

An unrecognized non-benign label raises an error at preprocessing time rather than silently defaulting to Benign (`attack_map.py`'s `stage_idx_for_label`) — every label that entered training is accounted for in this table, none were dropped silently. The Π transition matrix (Section 3.1) is counted directly over this same per-window single-stage label sequence; it should be read as a baseline/explanation aid over that simplified label stream, **not as a claim about ATT&CK's real non-linear structure**.

2.3 Feature schema

Each 60-second window is reduced to a flat 43-dimensional vector: flow volume/rate statistics, TCP flag rates, protocol shares, destination-port structure (entropy, well-known-service shares, distinct-port fraction — the scan signature), connection-shape signals (one-way/zero-byte fractions), and packet-level slots (TTL, fragmentation, retransmission, payload-size variance) — populated with real values where the source dataset supports it (CTU-13's real TTL) and zero-filled with an explicit flag otherwise (CIC-IDS-2018/2017's flow-CSV-only public release). No IP or port identity is ever fed to the model as a feature, only used as an aggregation key — a deliberate guard against the model memorizing “this IP is the attacker” instead of learning a transferable traffic signature.

The log1p fix

Heavy-tailed count/volume columns (flow count, byte/packet means) span orders of magnitude — a benign window might see `n_flows` $\approx 1,300$ while a DDoS window reaches `n_flows` $\approx 275,000$. Naive robust (median/IQR) scaling alone clips anything past ± 8 IQR-units to one identical saturated value, which erases exactly the signal that should most clearly mark an Impact-stage window. Applying $\log(1+x)$ before scaling compresses this range without destroying relative ordering. This was a concrete, verified bug fix, not a hypothetical concern: Impact-stage detection was materially better after the fix.

3. Core Architecture

3.1 Two-phase design

1. **Phase 1 — self-supervised dynamics.** An encoder maps the 43-dim window to a latent z_t ; a transition model produces a distribution $N(\mu, \sigma^2)$ over z_{t+1} (never a point estimate); a decoder reconstructs $\hat{S}_{t+1}$. Trained purely on Gaussian negative log-likelihood of the real next window, with **no label ever touched**.
2. **Phase 2 — supervised heads.** With the Phase-1 backbone optionally frozen (Section 4.4 covers when it should not be), a linear `StageHead` (7-way MITRE classification) and `InfilHead` (binary infiltration probability) are trained on top, plus a Laplace-smoothed, counted stage-transition matrix Π (never gradient-trained) for a cheap, interpretable closed-form forecast check: $\hat{y}_{t+K} = (\Pi^\top)^K y_t$.

3.2 The multi-step training fix

The dynamics loss is scored two ways, **concatenated** into one loss rather than weighted against each other:

- one-step, teacher-forced: real $z_{\leq t}$ in, score z_{t+1} against reality, at every position;
- multi-step, open-loop: encode only the context, roll the transition forward *consuming nothing but its own sampled output*, decode each imagined step, score against the real future observations.

Training on the one-step term alone never teaches the model to stay accurate over an open-loop rollout — it only ever sees real history during training, then is asked to imagine chains of its own predictions at inference. Adding the multi-step term (this is what “latent overshooting” means in the RSSM literature) closed nearly the entire real-vs-imagined accuracy gap for the two best-supported classes (Benign, Command & Control), while correctly leaving the three data-scarce classes untouched — confirming this was a training-regime fix, not a data-volume fix.

3.3 Infiltration probability is derived, not a dedicated head

Infiltration probability is the stage-classifier's own softmax mass on {Lateral Movement, Command & Control, Exfiltration}, not a separately-trained head. A dedicated `InfilHead` was built and tested twice, under two different class-weighting schemes, and was **consistently worse** than the derived approach both times — a clean, confirmed negative result, reported honestly rather than buried.

3.4 Evaluation scope: what is, and is not, measured

Every result in Sections 4–5 is a measurement of the **two downstream heads** (`StageHead`, `InfilHead`) built on top of the Phase-1 latent, not of the Phase-1 dynamics model in isolation. This distinction matters because a change that improves those heads — most notably unfreezing the encoder (Section 4.4) — is not automatically evidence that the underlying *learned network dynamics* improved; it may equally mean the representation was pulled toward whatever the classification/forecasting objective rewards, at some unmeasured cost to the dynamics model's own quality. Three concrete gaps follow from this, stated explicitly rather than left implicit:

- **No separate open-loop multi-step reconstruction metric is reported.** The multi-step training loss (Section 3.2) trains against exactly this quantity — Gaussian NLL of the decoder's imagined rollout against the real future window — but this report surfaces only the downstream Brier/AUC/F1 built on top of that latent, never the raw per-feature reconstruction error itself, frozen vs. unfrozen, side by side.
- **No future communication-structure/graph-level prediction is evaluated.** The two graph-structured encoder variants (Section 4.2) are evaluated only through the same two downstream heads as every other variant; neither this report nor the underlying project measures whether either graph encoder's own predicted future edge structure matches the real future communication pattern.
- **SHAP explains one window's classification, not a multi-step forecast rollout** (Section 7 states this boundary explicitly).

The system therefore supports a **narrower, more defensible claim** than “the world model’s dynamics improved”: it supports “the latent representation, combined with two supervised heads, produces a useful forecast and classification signal, whose quality is measured directly; whether the *unsupervised dynamics model in isolation* improved is not separately established here.” Readers should treat every comparison in Section 4 under that scope.

4. Architecture Search

Roughly a dozen variants were built and evaluated on identical splits, using two metrics throughout: **Brier score** (mean squared error of the predicted probability against the binary outcome — calibration) and **AUC** (probability a random true positive outranks a random true negative — ranking quality, ignoring calibration; 0.5 is random, below 0.5 is *worse* than random). Table 1 summarizes every architecture on pooled metrics; per-domain and per-class detail follows.

Table 1. Every architecture tried, pooled ("all domains") metrics, neural method.

Architecture	Brier (k1 → 5)	AUC (k1 → 5)	macroF1 (real/imag.)
Baseline Transformer (frozen)	0.140 → 0.128	0.913 → 0.911	0.376 / 0.302
RSSM, plain (frozen)	0.169 → 0.174	0.931 → 0.909	0.329 / 0.273
RSSM + domain-adversarial (frozen)	0.188 → 0.188	0.927 → 0.912	0.295 / 0.327
RSSM + feature-graph encoder (frozen)	0.124 → 0.141	0.955 → 0.931	0.386 / 0.306
KL-bottleneck-on-Transformer ablation	0.126 → 0.098	0.935 → 0.939	0.205 / 0.337
RSSM, encoder unfrozen	0.054 → 0.061	0.982 → 0.968	0.521 / 0.467
RSSM unfrozen + DANN (seed 1)	0.048 → 0.058	0.983 → 0.977	0.519 / 0.556
RSSM unfrozen + DANN (seed 2)	0.050 → 0.052	0.976 → 0.967	0.671 / 0.437
G-RSSM (host-graph, 2 of 3 domains)	0.174 → 0.181	0.775 → 0.765	0.418 / 0.469†

† Evaluated on a 3-of-7-class test set (only CTU-13 and CIC-IDS-2017, which lack several stages in their test slices) — not directly comparable to the 6-class pooled numbers above.

4.1 Domain-adversarial training (DANN)

Pooling three genuinely different network environments causes a real, measured failure: without correction, the stage classifier and Π work well on whichever domain dominates training and are actively worse than random on the others. DANN adds a small domain classifier trained *adversarially* against the shared encoder via a Gradient Reversal Layer (GRL): the domain head itself gets better at guessing which dataset a window came from, while the encoder underneath receives the *negated* gradient, pushed to make that guess harder. On the plain Transformer this helped the worst domain (CIC-IDS-2017) but hurt the previously-best one (CTU-13) — a genuine trade-off, not a clean win.

4.2 Two independent graph-structured encoders

Two structurally different graph ideas were tried, with markedly different outcomes:

Feature-level graph (GraFT-inspired)

Nodes are the 43 *features themselves* (not network hosts); edges are (a) fixed domain-knowledge groupings (e.g. all TCP-flag features connected to each other) and (b) per-window dynamic top-k cosine-similarity edges. A relational GCN with per-relation-type weights replaces the flat MLP encoder. Combined with the Transformer, this was **unstable** across epoch counts (AUC swinging from ≈ 0.5 to ≈ 0.07 between 30- and 80-epoch runs on CIC-IDS-2017) — consistent with the literature on GNN oversmoothing/initialization sensitivity, and reverted. Combined with RSSM instead, it was one of the **best** results of the whole project (best pooled AUC of any frozen variant, best frozen-model classification F1) — the instability was tied to the Transformer's own attention-based transition competing with a second attention mechanism inside the encoder, not an inherent flaw of the graph idea.

Host-communication graph (G-RSSM, arXiv:2604.14811)

Nodes are actual network hosts (IP addresses); edges are observed communication within a 60-second window; a per-host GRU plus masked cross-host attention replaces the flat encoder entirely. Only CTU-13 and CIC-IDS-2017 carry host identity (CIC-IDS-2018's public CSVs do not), so this variant covers two of three domains. The first implementation produced forecasts *systematically anti-correlated* with ground truth (AUC 0.13–0.30, below random) — traced to a real, identifiable bug: the top-16 hosts selected for each window were re-chosen *independently per window*, so a given “node slot” silently referred to a different physical host from one minute to the next, making the per-host recurrence meaningless. Fixing this (a day-level fixed host roster, one consistent slot per host for the whole day) genuinely repaired CIC-IDS-2017 forecasting (AUC 0.14 $\rightarrow$ 0.58, Π -check reaching 0.85) but left CTU-13 still broken (AUC 0.20–0.31) — a second, distinct, unresolved issue (likely that CTU-13's low-volume botnet hosts never make a naive top-16-by-volume cut).

4.3 The KL-bottleneck ablation: why does RSSM beat the Transformer?

RSSM's advantage over the Transformer could come from at least two different mechanisms: its GRU recurrence/smaller parameter count, or its stochastic posterior/prior plus free-bits KL regularization (the “bottleneck”). To isolate this, a small posterior head was added *on top of the otherwise-unchanged Transformer* — same backbone, same parameter count, no GRU — trained with DreamerV3's exact balanced free-bits KL against the Transformer's own prior. Result: the KL-bottleneck alone improved pooled Brier/AUC over the plain Transformer and won outright on CTU-13, but did **not** reproduce RSSM's CIC-IDS-2017 strength — so the GRU/recurrence itself (or seed variance) explains that specific domain's result, not the KL mechanism alone. The hypothesis splits cleanly rather than resolving to one cause.

4.4 Encoder freezing: the real bottleneck for classification

Every architecture above trains its StageHead on a **frozen** Phase-1 encoder — shaped only by the label-blind dynamics loss. A raw-feature logistic regression proved the seven stages are linearly

separable in the 43-dim input space (one-vs-rest AUC 0.90–0.999, including Reconnaissance with only 12 training examples, and Initial Access at 0.999), yet every frozen architecture scored ≤ 0.402 F1 on Initial Access and 0.000 on it for most variants. The mechanism: a frozen, label-blind encoder has no incentive to preserve a rare event's distinguishing detail if it barely moves the averaged dynamics loss — exactly the rate-distortion trade-off that makes rare, “surprising” states the most expensive thing a KL-regularized latent can encode faithfully.

Unfreezing the encoder for Phase 2 (letting the classification/infiltration gradient flow back into it) confirmed this decisively: macroF1 jumped $0.329 \rightarrow 0.521$, infiltration Brier improved roughly $3\times$, Initial Access F1 went from **0.000** to **0.402**, Impact from **0.208** to **0.628**. The cost: RSSM's own CIC-IDS-2017 domain-generalization strength partially collapsed (AUC $0.87 \rightarrow 0.65$), because fine-tuning on labels pulls the representation toward whatever the label-heavy domains need. Combining unfreezing with DANN recovered most of that loss on one seed (AUC up to 0.83) but the recovery **did not replicate** on a second seed (Section 4.5).

4.5 Seed variance: the discipline that mattered most

RSSM showed large run-to-run variance throughout this project. Two seeds of the same “unfrozen + DANN” configuration gave opposite answers to the specific question “does DANN recover RSSM's lost CIC-IDS-2017 strength”: seed 1 said yes (AUC $0.73 \rightarrow 0.83$), seed 2 said no (AUC $0.19 \rightarrow 0.30$, worse than a coin flip at $k=1$). Everything else about that configuration — pooled metrics, CIC-IDS-2018, CTU-13, classification F1 — replicated closely across both seeds. The honest conclusion: unfreezing and the general RSSM architecture are solid, replicated findings; the specific claim that DANN reliably rescues CIC-IDS-2017 domain generalization is **not established** and should not be reported as a settled result.

5. Per-Stage Forecasting Results

Binary infiltration forecasting collapses the full 7-way stage distribution into one number (probability mass on the three post-compromise stages) because it is the decision-relevant question for an operator (“will this become a serious breach”) and because several stages have too few test occurrences for a trustworthy per-horizon AUC. Table 2 makes this concrete, using **RSSM, encoder unfrozen** pooled across all three domains.

Table 2. Per-stage forecasting, pooled, derived/II AUC at $k=1$ and $k=5$.

Stage	n positive ($k=1 \rightarrow 5$)	AUC $k=1$	AUC $k=5$
Benign	711 $\rightarrow$ 731	0.960	0.951
Command and Control	989 $\rightarrow$ 977	0.987	0.974
Initial Access	17 $\rightarrow$ 13 (small)	0.979	0.999
Lateral Movement	14 $\rightarrow$ 10 (small)	0.947	0.903
Impact	5 $\rightarrow$ 5 (not trustworthy)	1.000	1.000

Stage	n positive (k1 → 5)	AUC k=1	AUC k=5
Reconnaissance	0 everywhere	insufficient test occurrences	
Exfiltration	0 everywhere	insufficient test occurrences	

Command-and-Control forecasting is the one genuinely large-sample, statistically solid result across every domain. Reconnaissance and Exfiltration cannot be forecast at all with the current data: there are zero test windows anywhere that transition into either stage within any 5-minute horizon — a data-scarcity floor, not an architecture limitation.

6. Baselines and the Cross-Episode Generalization Test

6.1 The logistic-regression baseline

A per-horizon logistic regression (five separate binary models, one per forecast horizon, trained on the flattened context window, class-weight balanced) was built as the mandated baseline and given every fair chance. For most of the architecture search, it was competitive with or dominant over the world model — a result reported honestly throughout rather than minimized, and the single strongest signal that architectural complexity was not obviously buying its keep until the final `RSSM unfrozen` configuration.

6.2 The memorization check

CIC-IDS-2018 contains exactly two Infiltration episodes (Wednesday-28-02-2018 and Thursday-01-03-2018, both mapped to Lateral Movement). Every LR number reported elsewhere in this project was obtained by pooling windows from *all* days (including both infiltration days) and testing on a later time slice of those same days — meaning the model had already partially seen the episode it was evaluated on. A true **episode-level** holdout (train the LR purely on one day, test purely on the other, never mixed) tells a very different story:

Table 3. True cross-episode holdout, logistic regression.

Direction	Brier (k1 → 5)	AUC (k1 → 5)
Train Wed-28 → Test Thu-01	0.182 → 0.220	0.792 → 0.824
Train Thu-01 → Test Wed-28	0.584 → 0.594	0.676 → 0.742

AUC collapses from the reported ≈ 0.995 (within-episode) to 0.68–0.82 (true holdout) — a substantial drop, though not to chance, meaning LR does learn some genuinely transferable signal, just far less than the in-distribution number suggested. The Thursday → Wednesday direction is worse than *useless* on Brier (0.58, worse than predicting the constant base rate), even though its AUC ranking is still reasonably above random — a real dissociation between ranking quality and calibration trustworthiness.

The same test on the world model (`RSSM unfrozen`, trained on everything *except* the held-out day) shows AUC 0.72–0.82 across both directions and, critically, **never** the catastrophic Brier failure LR

showed in one direction (Brier stays 0.21–0.31 both ways). Neither method has a decisive AUC edge on a genuinely unseen episode; the real differentiator is calibration stability, not ranking accuracy — a materially more honest and more useful claim for a predictive-defence system than “the model beats the baseline.”

7. Explainability

7.1 Attention and Π

Attention weights are read directly off the Transformer's own computation, not a post-hoc approximation, when that backbone is used — but reading an attention weight off a model is not, on its own, a guarantee that the weight is a *faithful* explanation of the model's actual computation (a well-documented general limitation of attention-as-explanation in the interpretability literature); it is offered here as a diagnostic signal, not a certified causal account. The Π transition matrix is directly interpretable by construction — every entry is a literal, counted transition probability, and the K-step forecast is a matrix power, not a learned function — subject to the scope stated in Section 2.2 (it is counted over the simplified single-stage-per-window label, not native ATT&CK ground truth).

7.2 SHAP

A `KernelExplainer` wraps the model's stage-classification output (holding the fixed prior context and perturbing only the most recent window's 43 raw features) and explains, per window, which features drove the **current, single-window** stage call. **This is explicitly not the same claim as explaining a multi-step (k=1..5) forecast rollout** — no SHAP attribution is computed against the rollout's own output, and none is claimed to exist here; extending this to a genuine multi-step-rollout explanation is a stated open item, not an implicit claim of this report. Two consistent, load-bearing findings about the single-window explanation that is computed:

- **SHAP magnitude tracks decision-boundary proximity, not a fixed scale.** On confidently-Benign windows, attributions are tiny (~0.0001–0.0005) because the softmax has little room to move; during a real, live-tested Lateral Movement attack, attributions grow roughly 30–100× larger (~0.004–0.016), correctly reflecting that the model is actually near a decision boundary.
- **Validated against a genuine attack in real time.** Replaying the actual Wednesday-28-02-2018 attack window-by-window (Section 8) showed infiltration probability climbing *one window before* the ground-truth label flipped to Lateral Movement (0.61–0.67 at the window immediately prior), then the stage classification itself correctly flipping to Lateral Movement at the true transition window and staying correct for the sustained 18-window attack duration, with SHAP consistently attributing the call to `svc_smtp_frac`, `svc_ssh_frac`, `half_open_frac`, and `bwd_pkts_mean`.

8. Working Prototype

A replay dashboard

Visualizes the model's *saved* predictions over a real attack day as an animated MITRE-stage strip, an infiltration-probability forecast chart, a scrubbable timeline, and per-window SHAP bars. Useful for demonstration and inspection, but — stated plainly in its own footer — it is a report of precomputed output, not a live tool.

A live-inference server (the actual PS deliverable)

A Flask backend (`server.py`) loads the trained model once at startup and exposes two endpoints: `POST /api/analyze` accepts an uploaded CSV (43-feature schema) or a raw PCAP file, extracts windows (with genuine packet-level TTL/fragmentation/payload-size features when given a PCAP, computed from real packets rather than zero-filled), and runs the world model *live* to return a forecast and stage classification for every window; `POST /api/explain` computes a SHAP explanation for one specific window on demand, so a full KernelExplainer pass is never paid for windows the user never looks at. A browser dashboard (drag-and-drop upload, or one-click real sample buttons for the two genuine CIC-IDS-2018 attack days) drives this API and renders the same visual language as the replay dashboard, but every number on screen was computed by the model for that specific input at request time.

9. Known Limitations

Reported plainly, as an inventory of open problems rather than a list of excuses:

- **Reconnaissance and Exfiltration are unforecastable and (Reconnaissance) barely classifiable with current data** — 12 total training examples for Reconnaissance, zero Exfiltration examples anywhere in the pooled corpus. No architecture change fixes a class the data does not contain.
- **CTU-13's host-graph forecasting (G-RSSM) remains broken** after fixing the node-identity bug; a second, distinct issue (likely host-selection criterion) is unresolved.
- **CICIoT2023 could not be integrated:** its public release has no timestamps and pre-shuffles packets into fixed-size groups, incompatible with time-windowed sequence forecasting at the source-format level, not a preprocessing choice.
- **The strongest single result (RSSM unfrozen + DANN) has an unreplicated headline claim.** Its domain-generalization recovery on CIC-IDS-2017 held on one seed and failed on a second; only the plain `RSSM, encoder unfrozen` configuration (without DANN) has fully replicated numbers across the seeds actually run.
- **Packet-level PCAPs for CIC-IDS-2018/2017 were not downloaded** (~220 GB); packet-level features (TTL, fragmentation, retransmission, payload-size variance) are zero-filled for those two datasets in the training pipeline.
- **The live-PCAP inference path has an unmeasured train-serving distribution mismatch.** When the live demo is given a real PCAP, those same packet-level fields are populated with genuine, non-zero values computed from real packets — values the model never observed during training on the

two zero-filled datasets (roughly two-thirds of the pooled training corpus by day count). Whether this shift degrades reliability has not been measured (e.g. via a held-out comparison of the model's calibration on zero-filled vs. genuinely-populated packet-level inputs); the live demo's few qualitative test runs are not a substitute for that measurement, and it should be treated as open rather than assumed benign.

10. Conclusion

The system addresses the problem statement's core requirements, within the scope this report states explicitly rather than implies: a latent-transition model (not a static classifier) built from a self-supervised dynamics objective supports downstream forecasting of attacker progression several minutes ahead and 7-way stage classification, maps predictions onto a MITRE-ATT&CK-derived taxonomy whose construction is documented in Section 2.2 rather than left implicit, and exposes two explanation channels (attention/ Π and single-window SHAP) whose claim boundaries are stated in Section 7. The one result that should be treated as the project's actual generalization claim is the cross-episode holdout (Section 6, $AUC \approx 0.68\text{--}0.82$ on two held-out episodes) — modest, above chance, and honestly small-sample. The strongest configuration on the (separately reported, in-distribution) pooled metrics, **RSSM with an unfrozen encoder**, replicates across five seeds on those metrics, but that result describes the two downstream heads measured, not an independently verified improvement in the dynamics model itself (Section 3.4). Equally important to the project's credibility is what is reported *without* spin: a plain logistic regression was competitive for most of the search, a dedicated infiltration head made things worse, two graph-encoder attempts each failed in different, diagnosable ways, one headline result (DANN's domain-generalization recovery) did not survive a second seed, several stage classes have too little test support for a confident claim, and the true cross-episode generalization ceiling is substantially lower than the in-distribution numbers on their own would imply. Every one of these negative or qualified findings was investigated to a specific, verifiable cause rather than smoothed over — which is, together with the working live-inference prototype, the actual contribution of this project.

Reproducibility

Provided here, stated honestly against what is **not** provided (this report is not a sealed reproducibility package):

- **Provided:** context length 16 windows, forecast horizon 5 windows; Phase 1 (dynamics) trained 30 epochs, Phase 2 (heads) 20 epochs, AdamW, weight decay $1e-4$, learning rate $3e-4$ (Phase 1) / $1e-3$ (Phase 2), gradient clipped to norm 1.0; RSSM dimensions $d_h=32$, $d_z=16$, $d_{\text{embed}}=64$; free-bits KL floor of 1 nat per direction (DreamerV3's exact balanced free-bits formulation); temporal train/val/test split 70/15/15 per source file, never crossing a day/scenario boundary; the five-seed replication of **RSSM, encoder unfrozen** reported in Section 4 used five independent Phase-1+Phase-2 training runs with no fixed random seed (relying on run-to-run variance itself, not a chosen seed list) and its full per-seed metrics.

- **Not provided:** exact train/val/test split manifests (row-level file lists), checkpoint file hashes, a frozen/sealed test protocol, or dataset-preprocessing content hashes. Reproducing the headline table from this report alone would require re-running the described pipeline against the same public dataset releases and accepting the same run-to-run variance documented in Section 4, not a byte-identical replay.

References

1. Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). *Mastering Diverse Domains through World Models*. arXiv:2301.04104. (DreamerV3; source of the exact balanced free-bits KL formulation used throughout this project's RSSM variants.)
2. Ganin, Y., & Lempitsky, V. (2015). *Unsupervised Domain Adaptation by Backpropagation*. Proceedings of ICML 2015. (Gradient Reversal Layer / domain-adversarial training, Section 4.1.)
3. Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). *Modeling Relational Data with Graph Convolutional Networks*. ESWC 2018. (Relational GCN formulation underlying the feature-level graph encoder, Section 4.2.)
4. AAAI'26 GraFT paper (ojs.aaai.org/index.php/AAAI/article/view/40029) — motivating reference for the feature-level relational graph encoder design.
5. Karacelebi, C., Sahin, Y. T., Surer, E., & Onur, E. *Learning Ad Hoc Network Dynamics via Graph-Structured World Models*. arXiv:2604.14811. (G-RSSM; per-host latent-state, cross-host-attention design underlying the host-communication graph encoder, Section 4.2.)
6. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. ICISSP 2018. (CIC-IDS-2018/2017, Canadian Institute for Cybersecurity.)
7. Garcia, S., Grill, M., Stiborek, J., & Zunino, A. (2014). *An Empirical Comparison of Botnet Detection Methods*. Computers & Security, 45, 100–123. (CTU-13, Stratosphere IPS Laboratory.)